NumPy

Python — Arrays, Vectorisation, Broadcasting & Array Operations

1	Fast array operations vs lists
2	Array creation — np.array()
3	Array homogeneity & dtype
4	np.arange() — generated arrays
5	Indexing & slicing
6	Reshaping — reshape() & flatten()
7	Filtering arrays
8	Special arrays — zeros, ones, full, eye, diag
9	Matrix multiplication — matmul / dot / @
10	Random numbers — np.random
11	np.linspace() & np.logspace()
12	Copy vs View — array() vs asarray()
13	Broadcasting rules
14	Transpose
15	np.repeat() — element repetition
16	np.tile() — series repetition
17	Stacking — vstack & hstack

1 · Fast Array Operations

- NumPy arrays are stored in a **continuous block of memory**, unlike Python lists.
- Operations on arrays are **vectorised** — applied element-wise without Python loops.
- Lists with ***** operator replicate the list itself (e.g. `[1,2] * 3` → `[1,2,1,2,1,2]`), whereas arrays perform element-wise multiplication.
- NumPy operations are implemented in C under the hood, making them orders of magnitude faster than pure Python loops.

2 · Array Creation — np.array()

```
import numpy as np

array1 = np.array([1, 2, 3, 4, 5])
array1 = np.array([1, 2, 3, 9.8, 9], dtype=float)  # specify dtype
```

3 · Array Homogeneity & dtype

- Arrays are **homogeneous** — all elements must be of the same data type.
- If mixed types are provided, NumPy upcasts automatically (e.g. int + float → float64).
- Use the `dtype` parameter to force a specific type: `np.array([1,2,3], dtype=float)`.

4 · np.arange() — Generated Arrays

Generates arrays similar to Python's `range(start, stop, step)`, but returns a NumPy array stored in **continuous block memory**.

```
np.arange(start, stop, step)

np.arange(0, 10, 2)   # array([0, 2, 4, 6, 8])
np.arange(5)           # array([0, 1, 2, 3, 4])
```

■ np.arange() is equivalent to range() but produces a NumPy array. Use np.linspace() when you need a specific number of evenly-spaced values rather than a fixed step size.

5 · Indexing & Slicing

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])

arr[0]      # whole first row  → [1, 2, 3]
arr[0][2]   # specific element → 3   (row 0, col 2)
arr[1, 3]   # same as arr[1][3] using comma notation

# Slicing: arr[start:stop:step]
arr[0:2]    # first two rows
```

6 · Reshaping — reshape() & flatten()

Convert between 1-D and multi-dimensional arrays.

```
arr = np.arange(6)           # [0, 1, 2, 3, 4, 5]

# reshape to 2-D
arr.reshape(2, 3)
# array([[0, 1, 2],
#        [3, 4, 5]])

# Use -1 when you don't know one dimension
arr1 = arr.reshape(-1, 1)    # NumPy infers the row count

# ravel() — returns a VIEW (changes affect original)
original_array.ravel()

# flatten() — returns a COPY (safe to modify independently)
original_array.flatten()
```

Method	Returns	Modifies Original?
reshape()	Reshaped view / copy	Yes — it's a view
ravel()	1-D view	Yes — it's a view
flatten()	1-D copy	No — independent copy

7 · Filtering Arrays (Boolean Indexing)

Pass a boolean condition inside square brackets to filter elements.

```
arr2 = np.array([1, 2, 3, 4, 5, 6])

# Keep only even numbers
arr2[arr2 % 2 == 0]   # array([2, 4, 6])

# Condition produces a boolean mask first:
# arr2 % 2 == 0  →  [False, True, False, True, False, True]
```

■ Boolean indexing always returns a copy, not a view. Assign back to the original array if you want to modify it in place.

8 · Special Arrays — zeros, ones, full, eye, diag

```
np.zeros((3, 4))          # 3×4 array of 0.0
np.ones((1, 5))           # 1×5 array of 1.0
np.full((4, 5), 11)       # 4×5 array filled with 11

np.eye(3)                 # 3×3 identity matrix
np.diag([2, 3, 4, 5])    # diagonal matrix with given values
```

Function	Description
np.zeros(shape)	Array of zeros
np.ones(shape)	Array of ones
np.full(shape, val)	Array filled with a constant value

np.eye(n)	n×n identity matrix — 1s on diagonal, 0s elsewhere
np.diag([v1,v2,…])	Diagonal matrix from given values

9 · Matrix Multiplication

Three equivalent ways to perform matrix multiplication in NumPy:

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

np.matmul(A, B)   # recommended for matrices
np.dot(A, B)      # works for 1-D and 2-D
A @ B             # Python operator (PEP 465) — most concise
```

■ All three produce the same result for 2-D arrays. Prefer @ for clarity. np.matmul() does not support scalar multiplication, whereas np.dot() does.

10 · Random Numbers — np.random

```
# Any integer between 1 and 100
np.random.randint(1, 100)

# 5 random integers between 1 and 100
np.random.randint(1, 100, 5)

# Random floats between 0 and 1
np.random.rand()           # single value
np.random.rand(3, 2)       # 3x2 array

# Random floats from standard normal distribution (mean=0, std=1)
np.random.randn(3, 2)
```

Function	Output	Range
np.random.randint(lo, hi)	Random integers	lo to hi-1
np.random.rand()	Random float(s)	0.0 to 1.0 (uniform)
np.random.randn()	Random float(s)	Normal distribution μ=0, σ=1

11 · np.linspace() & np.logspace()

```
# linspace: evenly-spaced values (endpoints inclusive by default)
np.linspace(0, 10, 5)    # [0., 2.5, 5., 7.5, 10.]
                          # ^ start  ^ stop  ^ num of values

# logspace: evenly-spaced on log scale
np.logspace(0, 3, 4)     # [1., 10., 100., 1000.]
```

Function	Use when…
np.arange(start, stop, step)	You know the step size
np.linspace(start, stop, n)	You know the number of points you want

np.logspace(start, stop, n)	You need logarithmically-spaced points

12 · Copy vs View — array() vs asarray()

```
arr = np.array([1, 2, 3])

a = np.array(arr)    # False copy  — independent (modifying a won't affect arr)
b = np.asarray(arr)  # True  view  — shares data  (modifying b WILL affect arr)

print(a is arr)      # False
print(b is arr)      # True  (same memory)
```

■ Use np.array() when you need a safe independent copy. Use np.asarray() when you want to avoid unnecessary memory duplication and know you won't modify the data.

13 · Broadcasting

Broadcasting allows NumPy to perform arithmetic on arrays of different shapes by automatically expanding the smaller array along the missing/size-1 dimensions.

Broadcasting rules:

- Either **1** or the **other** dimension should be the same.
- **Both** dimensions are the same — works directly.
- One dimension is the same and the other is **unknown (1)** — NumPy stretches it.

```
a = np.array([[1,2,3],   # shape (2,3)
              [4,5,6]])
b = np.array([10,20,30]) # shape (3,)   → broadcast to (2,3)

a + b
# array([[11, 22, 33],
#        [14, 25, 36]])
```

■ If dimensions are incompatible and neither is 1, NumPy raises a ValueError. Always check array shapes with arr.shape before operations.

14 · Transpose

```
arr = np.array([[1,2,3],
                [4,5,6]])   # shape (2,3)

arr.T                       # shape (3,2)  — rows become columns

# array([[1, 4],
#        [2, 5],
#        [3, 6]])
```

■ You cannot transpose a 1-D array — it must have at least 2 dimensions (rows and columns). For a 1-D array, use arr.reshape(1, -1).T to convert it first.

15 · np.repeat() — Element Repetition

Repeats each element a given number of times along a specified axis.

```
arr2 = np.array([[1,2],
                 [3,4]])

np.repeat(arr2, 2, axis=1)
# array([[1, 1, 2, 2],
#        [3, 3, 4, 4]])

# axis=0 repeats rows; axis=1 repeats columns (elements along axis)
```

■ np.repeat() repeats individual elements. Compare with np.tile() which repeats the whole array (like tiling a floor).

16 · np.tile() — Series Repetition

```
arr = np.array([1, 2, 3])

np.tile(arr, 3)
# array([1, 2, 3, 1, 2, 3, 1, 2, 3])

# Repeats the ENTIRE series, not individual elements
```

Function	Behaviour	Example
np.repeat(a, n)	Each element repeated n times	[1,2] → [1,1,2,2]
np.tile(a, n)	Whole array repeated n times	[1,2] → [1,2,1,2]

17 · Stacking — vstack & hstack

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Vertical stack — adds rows (stacks along axis 0)
np.vstack((a, b))
# array([[1, 2, 3],
#        [4, 5, 6]])

# Horizontal stack — adds columns (stacks along axis 1)
np.hstack((a, b))
# array([1, 2, 3, 4, 5, 6])
```

Function	Direction	Result shape (for 1-D inputs)
np.vstack((a,b))	Vertical (↓ rows)	(2, n) — 2-D array
np.hstack((a,b))	Horizontal (→ columns)	(2n,) — 1-D array

■ For 2-D arrays, vstack requires matching column counts; hstack requires matching row counts. np.concatenate() is the generalised version for any axis.

Quick Reference — NumPy Essentials

Tool / Method	Purpose	Import needed?
np.array()	Create an array from a list	import numpy as np

np.arange()	Create range-based array	No (after import)
np.linspace()	n evenly-spaced values	No
np.zeros/ones/full()	Create constant-value arrays	No
np.eye()	Identity matrix	No
arr.reshape()	Change shape	No
arr.flatten()	Collapse to 1-D (copy)	No
arr.ravel()	Collapse to 1-D (view)	No
arr.T	Transpose (swap axes)	No
np.matmul() / @	Matrix multiplication	No
np.vstack/hstack()	Stack arrays vertically / horizontally	No
np.repeat()	Repeat elements	No
np.tile()	Repeat whole array	No
np.random.rand()	Uniform random floats [0,1)	No
np.random.randint()	Random integers in range	No

```
# Classic functional pipeline with NumPy
import numpy as np

arr = np.arange(1, 11)              # [1..10]
evens = arr[arr % 2 == 0]           # filter evens
squared = evens ** 2                # vectorised square
total = squared.sum()               # reduce to scalar
print(total)                        # 220
```

■ This pipeline — create → filter → transform → reduce — mirrors the map/filter/reduce pattern in functional programming, but exploits NumPy's vectorisation for speed.